

Week 7 – Locating Files on the Linux System

Topics covered

- Filesystem Hierarchy Standard
 - Finding files with `locate`
 - Updating the `locate` database
 - Real-time searches using `find`
 - Searching by name, type, age, size and ownership
 - Performing actions on results with `-exec`
 - Locating commands with `whereis` and `which`
 - Lab: RH124 Chapter 13.7 – Access Linux File System
-

1. Filesystem Hierarchy Standard

The Filesystem Hierarchy Standard, or FHS, defines the expected names and purposes of directories in a Linux system.

Linux uses an inverted tree structure beginning at:

`/`

Important directories from the Week 7 slides include:

Directory	Purpose
<code>/</code>	Root of the main filesystem hierarchy
<code>/bin</code>	Essential user executable commands
<code>/boot</code>	Kernel and bootloader files

<code>/dev</code>	Files representing attached devices
<code>/etc</code>	Host-specific configuration files
<code>/home</code>	Regular users' home directories
<code>/opt</code>	Optional third-party software
<code>/root</code>	Root user's home directory
<code>/run</code>	Volatile runtime data
<code>/sbin</code>	Essential administrative executables
<code>/srv</code>	Data provided by system services
<code>/sys</code>	Kernel and hardware information
<code>/tmp</code>	Temporary files
<code>/usr</code>	Secondary software hierarchy
<code>/usr/bin</code>	Non-essential user commands

<code>/usr/lib</code>	Shared program libraries
<code>/usr/local</code>	Locally installed software
<code>/usr/sbin</code>	Non-essential administrative commands
<code>/usr/share</code>	Architecture-independent data
<code>/var</code>	Logs, web data, mailboxes and changing data

The diagram on page 5 illustrates these directories branching downward from `/`.

2. Finding files with `locate`

Basic syntax:

```
locate search_term
```

Example:

```
locate passwd
```

This displays paths containing the word `passwd`, such as:

1. `/etc/passwd`
2. `/usr/bin/passwd`

```
/usr/share/man/man1/passwd.1.gz
```

The search term may appear anywhere in the path or filename.

3. How `locate` works

locate does not search the filesystem directly. It searches a previously generated database of file paths.

Advantages:

- Very fast
- Useful for broad searches across the system

Disadvantages:

- Results may be outdated
- Newly created files may not appear
- Deleted files may still appear until the database is updated

The command only returns results that the current user is permitted to access.

4. Case-sensitive and case-insensitive searches

Normal searches are case-sensitive:

```
locate bashrc
```

Case-insensitive search:

```
locate -i bashrc
```

This can match names such as:

3. .bashrc
4. BASHRC

```
Bashrc
```

The **-r** option allows a regular-expression search:

```
locate -r '\.conf$'
```

This searches for paths ending in **.conf**.

5. Updating the **locate** database

Update the database manually:

```
sudo updatedb
```

Example workflow from the slides:

```
5. touch lostfile
```

```
locate lostfile
```

The newly created file may not appear.

Update the database:

```
6. sudo updatedb
```

```
locate lostfile
```

Expected result:

```
/home/student/lostfile
```

Delete it:

```
7. rm lostfile
```

```
locate lostfile
```

The deleted file may still appear because the database has not been refreshed again.

Update once more:

```
8. sudo updatedb
```

```
locate lostfile
```

The result should disappear. This demonstrates why **locate** is fast but potentially outdated.

6. **locate** database configuration

The database configuration file is:

```
/etc/updatedb.conf
```

It controls which filesystems, directories or path types may be excluded from the database.

The database is also normally refreshed automatically through a scheduled system task.

7. Finding files with **find**

Unlike **locate**, **find** searches the actual filesystem in real time.

Basic syntax:

```
find STARTING_DIRECTORY SEARCH_CRITERIA
```

Search the current directory:

```
find .
```

Search the current user's home directory:

```
find ~
```

Search the entire system:

```
find /
```

Because a full-system search may produce permission errors, use:

```
find / 2>/dev/null
```

The slides explain that **find** is slower than **locate**, but its results reflect what currently exists.

8. Searching by exact name

Find a file named **passwd** under **/etc**:

```
find /etc -name passwd
```

Find files ending in **.conf**:

```
find /etc -name "*.conf"
```

The quotation marks prevent the shell from expanding ***.conf** before **find** processes it.

9. Case-insensitive name search

```
find /home -iname "report.txt"
```

This may match:

- 9. report.txt
- 10. Report.txt

REPORT.TXT

Difference:

Option	Meaning
<code>-name</code>	Case-sensitive
<code>-iname</code>	Case-insensitive

10. Searching by file type

Find regular files:

```
find /etc -type f
```

Find directories:

```
find /usr -type d
```

Common types:

Type	Meaning
<code>f</code>	Regular file

d Directory

l Symbolic link

Find symbolic links:

```
find /usr -type l
```

11. Limiting search depth

Search only the given directory without entering subdirectories:

```
find /etc -maxdepth 1
```

Find only `.conf` files directly under `/etc`:

```
find /etc -maxdepth 1 -type f -name "*.conf"
```

Without `-maxdepth`, `find` recursively searches all subdirectories.

12. Searching by modification time

Find files modified within the last three days:

```
find / -mtime -3 2>/dev/null
```

Meaning of values:

Expression	Meaning
<code>-mtime -3</code>	Modified fewer than three days ago

`-mtime 3`

Modified approximately three days ago

`-mtime +3`

Modified more than three days ago

Search by minutes:

```
find /tmp -mmin -10
```

This finds files modified within the last ten minutes.

The Week 7 slides specifically list `-mtime -3` and `-mmin -10`.

13. Searching by size

Find files larger than 1 MB:

```
find / -size +1M 2>/dev/null
```

Find files larger than 20 KB:

```
find . -size +20k
```

Useful size suffixes:

Suffix	Meaning
<code>c</code>	Bytes
<code>k</code>	Kilobytes
<code>M</code>	Megabytes

G

Gigabytes

Examples:

```
11. find /var/log -size +5M
```

```
find /home -size -100k
```

- **+5M** means larger than 5 MB.
 - **-100k** means smaller than 100 KB.
-

14. Searching by owner

Find files owned by user **joe**:

```
find /home -user joe
```

Find files whose owner account no longer exists:

```
find / -nouser 2>/dev/null
```

Find files whose group no longer exists:

```
find / -nogroup 2>/dev/null
```

The slides explicitly include **-user joe** and **-nouser**.

15. Finding empty files and directories

Find all empty files and directories:

```
find /home -empty
```

Only empty files:

```
find /home -type f -empty
```

Only empty directories:

```
find /home -type d -empty
```

16. Combining conditions with AND

When several conditions are placed together, `find` treats them as a logical AND.

Example:

```
find /var/log -type f -size +5M
```

This finds entries that are:

- Regular files
- Larger than 5 MB

Another example:

```
find /home -type f -user student -name "*.txt"
```

The file must satisfy all three conditions.

17. Combining conditions with OR

Use:

```
-o
```

Example:

```
find . -name "*.txt" -o -name "*.log"
```

This finds names ending in either `.txt` or `.log`.

Parentheses make the logic clearer:

```
find . \( -name "*.txt" -o -name "*.log" \)
```

The parentheses must be escaped so that the shell does not process them.

The slides explain that multiple criteria normally use AND, while `-o` creates an OR condition.

18. Performing actions with `-exec`

`-exec` runs a command on each item found.

Display detailed information about each `.conf` file:

```
find /etc -name "*.conf" -exec ls -l {} \;
```

Meaning:

P	Purpose
-	Run a command
l	Command to execute
{ }	Placeholder for each result
\	Marks the end of the command

The slides specifically describe `{ }` as the placeholder for each found filename.

Another example:

```
find /tmp -type f -empty -exec ls -l {} \;
```

19. Deleting files found by **find**

Review the results first:

```
find /tmp -type f -name "*.tmp"
```

Then delete them:

```
find /tmp -type f -name "*.tmp" -delete
```

This is a valid **find** action, but the Week 7 slides mainly focus on **-exec**. Use deletion commands carefully.

20. Finding commands with **whereis**

whereis searches for:

- Command binaries
- Source files
- Manual pages

Example:

```
whereis grep
```

Possible output:

```
grep: /usr/bin/grep /usr/share/man/man1/grep.1.gz
```

Locate **ls**:

```
whereis ls
```

Possible output:

```
ls: /usr/bin/ls /usr/share/man/man1/ls.1.gz
```

21. Finding the command that will execute with **which**

Use:

which grep

Possible output:

/usr/bin/grep

which searches the directories listed in the current user's **PATH** and shows the executable that the shell would use.

Check the path variable:

echo \$PATH

22. **whereis** versus **which**

Comm an d	Purpose
where is	Finds binaries, manuals and possibly source files
which	Shows the executable selected from PATH
locate e	Searches a file-path database
find	Searches the current filesystem in real time

23. Finding multiple executable versions

Display all matching executables in **PATH**:

which -a passwd

Possible output:

12. /usr/bin/passwd

/bin/passwd

Depending on the system, `/bin` may be a symbolic link to `/usr/bin`, so these may not represent separate physical programs.

The slides mention that `which -a` can help identify whether an unexpected executable has been inserted earlier in `PATH`.

Week 7 – Locating Files on the Linux System

Topics covered

- Filesystem Hierarchy Standard
- Finding files with `locate`
- Updating the `locate` database
- Real-time searches using `find`
- Searching by name, type, age, size and ownership
- Performing actions on results with `-exec`
- Locating commands with `whereis` and `which`
- Lab: RH124 Chapter 13.7 – Access Linux File System

1. Filesystem Hierarchy Standard

The Filesystem Hierarchy Standard, or FHS, defines the expected names and purposes of directories in a Linux system.

Linux uses an inverted tree structure beginning at:

/

Important directories from the Week 7 slides include:

Directory	Purpose
-----------	---------

<code>/</code>	Root of the main filesystem hierarchy
<code>/bin</code>	Essential user executable commands
<code>/boot</code>	Kernel and bootloader files
<code>/dev</code>	Files representing attached devices
<code>/etc</code>	Host-specific configuration files
<code>/home</code>	Regular users' home directories
<code>/opt</code>	Optional third-party software
<code>/root</code>	Root user's home directory
<code>/run</code>	Volatile runtime data
<code>/sbin</code>	Essential administrative executables
<code>/srv</code>	Data provided by system services
<code>/sys</code>	Kernel and hardware information

<code>/tmp</code>	Temporary files
<code>/usr</code>	Secondary software hierarchy
<code>/usr/bin</code>	Non-essential user commands
<code>/usr/lib</code>	Shared program libraries
<code>/usr/local</code>	Locally installed software
<code>/usr/sbin</code>	Non-essential administrative commands
<code>/usr/share</code>	Architecture-independent data
<code>/var</code>	Logs, web data, mailboxes and changing data

The diagram on page 5 illustrates these directories branching downward from `/`.

2. Finding files with `locate`

Basic syntax:

```
locate search_term
```

Example:

```
locate passwd
```

This displays paths containing the word `passwd`, such as:

13. /etc/passwd

14. /usr/bin/passwd

/usr/share/man/man1/passwd.1.gz

The search term may appear anywhere in the path or filename.

3. How **locate** works

locate does not search the filesystem directly. It searches a previously generated database of file paths.

Advantages:

- Very fast
- Useful for broad searches across the system

Disadvantages:

- Results may be outdated
- Newly created files may not appear
- Deleted files may still appear until the database is updated

The command only returns results that the current user is permitted to access.

4. Case-sensitive and case-insensitive searches

Normal searches are case-sensitive:

```
locate bashrc
```

Case-insensitive search:

```
locate -i bashrc
```

This can match names such as:

15. .bashrc

16. BASHRC

Bashrc

The **-r** option allows a regular-expression search:

```
locate -r '\.conf$'
```

This searches for paths ending in `.conf`.

5. Updating the `locate` database

Update the database manually:

```
sudo updatedb
```

Example workflow from the slides:

```
17. touch lostfile
```

```
locate lostfile
```

The newly created file may not appear.

Update the database:

```
18. sudo updatedb
```

```
locate lostfile
```

Expected result:

```
/home/student/lostfile
```

Delete it:

```
19. rm lostfile
```

```
locate lostfile
```

The deleted file may still appear because the database has not been refreshed again.

Update once more:

```
20. sudo updatedb
```

```
locate lostfile
```

The result should disappear. This demonstrates why `locate` is fast but potentially outdated.

6. **locate** database configuration

The database configuration file is:

```
/etc/updatedb.conf
```

It controls which filesystems, directories or path types may be excluded from the database.

The database is also normally refreshed automatically through a scheduled system task.

7. Finding files with **find**

Unlike **locate**, **find** searches the actual filesystem in real time.

Basic syntax:

```
find STARTING_DIRECTORY SEARCH_CRITERIA
```

Search the current directory:

```
find .
```

Search the current user's home directory:

```
find ~
```

Search the entire system:

```
find /
```

Because a full-system search may produce permission errors, use:

```
find / 2>/dev/null
```

The slides explain that **find** is slower than **locate**, but its results reflect what currently exists.

8. Searching by exact name

Find a file named **passwd** under **/etc**:

```
find /etc -name passwd
```

Find files ending in `.conf`:

```
find /etc -name "*.conf"
```

The quotation marks prevent the shell from expanding `*.conf` before `find` processes it.

9. Case-insensitive name search

```
find /home -iname "report.txt"
```

This may match:

21. report.txt

22. Report.txt

REPORT.TXT

Difference:

Option	Meaning
<code>-name</code>	Case-sensitive
<code>-iname</code>	Case-insensitive

10. Searching by file type

Find regular files:

```
find /etc -type f
```

Find directories:

```
find /usr -type d
```

Common types:

Type	Meaning
f	Regular file
d	Directory
l	Symbolic link

Find symbolic links:

```
find /usr -type l
```

11. Limiting search depth

Search only the given directory without entering subdirectories:

```
find /etc -maxdepth 1
```

Find only `.conf` files directly under `/etc`:

```
find /etc -maxdepth 1 -type f -name "*.conf"
```

Without `-maxdepth`, `find` recursively searches all subdirectories.

12. Searching by modification time

Find files modified within the last three days:

```
find / -mtime -3 2>/dev/null
```

Meaning of values:

Expression

Meaning

`-mtime -3`

Modified fewer than three days ago

`-mtime 3`

Modified approximately three days ago

`-mtime +3`

Modified more than three days ago

Search by minutes:

```
find /tmp -mmin -10
```

This finds files modified within the last ten minutes.

The Week 7 slides specifically list `-mtime -3` and `-mmin -10`.

13. Searching by size

Find files larger than 1 MB:

```
find / -size +1M 2>/dev/null
```

Find files larger than 20 KB:

```
find . -size +20k
```

Useful size suffixes:

Suffix

Meaning

`c`

Bytes

k Kilobytes

M Megabytes

G Gigabytes

Examples:

23. `find /var/log -size +5M`

`find /home -size -100k`

- **+5M** means larger than 5 MB.
- **-100k** means smaller than 100 KB.

14. Searching by owner

Find files owned by user **joe**:

`find /home -user joe`

Find files whose owner account no longer exists:

`find / -nouser 2>/dev/null`

Find files whose group no longer exists:

`find / -nogroup 2>/dev/null`

The slides explicitly include **-user joe** and **-nouser**.

15. Finding empty files and directories

Find all empty files and directories:

`find /home -empty`

Only empty files:

```
find /home -type f -empty
```

Only empty directories:

```
find /home -type d -empty
```

16. Combining conditions with AND

When several conditions are placed together, `find` treats them as a logical AND.

Example:

```
find /var/log -type f -size +5M
```

This finds entries that are:

- Regular files
- Larger than 5 MB

Another example:

```
find /home -type f -user student -name "*.txt"
```

The file must satisfy all three conditions.

17. Combining conditions with OR

Use:

```
-o
```

Example:

```
find . -name "*.txt" -o -name "*.log"
```

This finds names ending in either `.txt` or `.log`.

Parentheses make the logic clearer:

```
find . \( -name "*.txt" -o -name "*.log" \)
```

The parentheses must be escaped so that the shell does not process them.

The slides explain that multiple criteria normally use AND, while `-o` creates an OR condition.

18. Performing actions with `-exec`

`-exec` runs a command on each item found.

Display detailed information about each `.conf` file:

```
find /etc -name "*.conf" -exec ls -l {} \;
```

Meaning:

Part	Purpose
<code>-exec</code>	Run a command
<code>ls -l</code>	Command to execute
<code>{}</code>	Placeholder for each result
<code>\\;</code>	Marks the end of the command

The slides specifically describe `{}` as the placeholder for each found filename.

Another example:

```
find /tmp -type f -empty -exec ls -l {} \;
```

19. Deleting files found by `find`

Review the results first:

```
find /tmp -type f -name "*.tmp"
```

Then delete them:

```
find /tmp -type f -name "*.tmp" -delete
```

This is a valid **find** action, but the Week 7 slides mainly focus on **-exec**. Use deletion commands carefully.

20. Finding commands with **whereis**

whereis searches for:

- Command binaries
- Source files
- Manual pages

Example:

```
whereis grep
```

Possible output:

```
grep: /usr/bin/grep /usr/share/man/man1/grep.1.gz
```

Locate **ls**:

```
whereis ls
```

Possible output:

```
ls: /usr/bin/ls /usr/share/man/man1/ls.1.gz
```

21. Finding the command that will execute with **which**

Use:

```
which grep
```

Possible output:

```
/usr/bin/grep
```

`which` searches the directories listed in the current user's `PATH` and shows the executable that the shell would use.

Check the path variable:

```
echo $PATH
```

22. `whereis` versus `which`

Command	Purpose
<code>whereis</code>	Finds binaries, manuals and possibly source files
<code>which</code>	Shows the executable selected from <code>PATH</code>
<code>locate</code>	Searches a file-path database
<code>find</code>	Searches the current filesystem in real time

23. Finding multiple executable versions

Display all matching executables in `PATH`:

```
which -a passwd
```

Possible output:

```
24. /usr/bin/passwd
```

```
/bin/passwd
```

Depending on the system, `/bin` may be a symbolic link to `/usr/bin`, so these may not represent separate physical programs.

The slides mention that `which -a` can help identify whether an unexpected executable has been inserted earlier in `PATH`.

25.